

Recommended Monitoring Stack

Prometheus, Alertmanager, and Grafana

Node Exporter and cAdvisor

Blackbox Exporter

Loki and Grafana Alloy

Service-specific Prometheus exporters

Prometheus, Alertmanager, Grafana, Loki, and Blackbox Exporter should preferably run on a separate monitoring server. Node Exporter, cAdvisor, Grafana Alloy, and application-specific exporters should run on the Docker application host.

Resilience: If the Docker application host becomes unavailable, the monitoring platform must remain operational and generate an alert.

1. External Website Monitoring

Use Prometheus Blackbox Exporter to monitor the existing public WordPress homepage:

https://example.com/	
Area	Monitor
Connectivity	DNS resolution, TCP connectivity, TLS negotiation, and redirects
Availability	HTTP response status and expected homepage content
Performance	Total response time and individual request phases
Certificate	TLS certificate expiration

The probe can require HTTP 200 and search for a stable page title or another known string. Blackbox Exporter should run outside the Docker application host so it can detect complete server, Docker Engine, firewall, reverse proxy, DNS, certificate, or network failures.

Availability alert

```
probe_success{job="wordpress-public"} == 0  
for: 2m
```

Certificate expiration

```
(probe_ssl_earliest_cert_expiry - time()) < 14 * 86400
```

- Warning: fewer than 14 days remaining
- Critical: fewer than 3 days remaining

2. Docker Host Monitoring

Run Node Exporter on the Docker host.

Area	Monitor
Compute	CPU utilization, available memory, and load average
Storage	Filesystem capacity, disk I/O, and disk latency

Area	Monitor
Network	Network traffic and errors
System	File descriptors, uptime, and time synchronization

Recommended alerts

- Node Exporter unavailable
- CPU utilization above 90% for 10 minutes
- Available memory below 10%
- Filesystem below 15% free space; critical below 5%
- Filesystem predicted to fill within 24 hours
- High disk I/O latency or increasing network errors
- Unexpected server restart or unsynchronized system clock

Storage focus: Pay particular attention to the filesystems containing the Docker data directory, MySQL data, WordPress files, and any locally stored Prometheus or Loki data.

3. Docker Container Monitoring

Run cAdvisor as a Docker Compose service to collect resource metrics for running containers.

Area	Monitor
Compute	Container CPU usage, CPU throttling, memory working set, and memory limits
Lifecycle	Out-of-memory events, container start time, and last observed time
Storage	Container filesystem usage
Network	Container network traffic and errors

Dashboard scope

- Traefik and Varnish
- WordPress, NGINX, and PHP-FPM
- MySQL and Memcached
- Monitoring exporters

Recommended alerts

- cAdvisor unavailable or an expected Compose service absent
- Container memory usage approaching its limit
- Sustained CPU throttling
- Out-of-memory event detected
- Container filesystem usage increasing rapidly
- Container restarting repeatedly

Example missing-service alert using the Docker Compose label:

```
absent(
  container_last_seen{
    container_label_com_docker_compose_service="wordpress"
  }
)
```

Important: cAdvisor primarily reports running containers. A stopped container may disappear from its metrics, so missing-series alerts using `absent()` are important.

4. Docker Engine Monitoring

Docker Engine can optionally expose native Prometheus metrics through `/etc/docker/daemon.json`:

```
{
  "metrics-addr": "127.0.0.1:9323"
}
```

These metrics monitor Docker Engine itself, not WordPress or individual application containers. They can help detect Docker daemon availability, engine operations, builder activity, and lifecycle operation failures.

Supplementary signal: Docker notes that the available metric names may change. Treat Docker Engine metrics as supplementary rather than as the primary application monitoring source.

5. Traefik Monitoring

Enable Traefik's native Prometheus metrics.

Area	Monitor
Traffic	Request rate and HTTP 2xx, 4xx, and 5xx responses
Latency	Request duration, including p95 and p99
Connectivity	Open connections, routers, services, and backend errors
Configuration	Configuration reload failures

Recommended alerts

- Traefik metrics target unavailable
- HTTP 5xx rate above 2% for 5 minutes
- p95 latency above 2 seconds for 10 minutes
- Varnish backend reported as unavailable
- Increasing connection errors or configuration reload failures

Alert quality: Include a minimum request volume in error-rate alerts so one failed request does not produce a misleading 100% error rate.

6. MySQL Monitoring

Run `mysqld_exporter` as a separate Docker Compose service using a dedicated read-only monitoring account.

Area	Monitor
Availability	Exporter availability, MySQL availability, and uptime
Connections	Active connections, maximum connection usage, and threads running
Queries	Query rate, slow queries, deadlocks, lock waits, and temporary disk tables
InnoDB	Buffer pool use and related storage behavior
Storage/replication	Database storage and replication lag when applicable

Recommended alerts

- MySQL exporter or MySQL unavailable
- Connections above 80% of the configured limit
- Deadlocks detected or sustained increase in slow queries
- Database storage approaching capacity

- Replication lag above the accepted threshold

7. Memcached Monitoring

Run a maintained Memcached Prometheus exporter as a separate Compose service.

Area	Monitor
Availability	Exporter and Memcached availability
Capacity	Current connections, memory usage, and configured memory limit
Cache efficiency	Evictions, hits, misses, hit ratio, and command rate

Recommended alerts

- Memcached exporter or Memcached unavailable
- Memory usage above 85%
- Sustained cache evictions or connection exhaustion
- Significant reduction in cache hit ratio

Cache hit ratio alerts should be introduced only after establishing a normal production baseline.

8. Varnish Monitoring

Use the standard varnishstat counters through a Varnish Prometheus exporter.

- Cache hits, misses, and cache hit ratio
- Backend health, connection failures, and fetch failures
- Cache storage usage
- Thread pool usage and queued requests
- Dropped requests and ban operations

Recommended alerts

- Varnish exporter unavailable or WordPress backend unavailable
- Backend fetch failures increasing
- Requests queued because of thread exhaustion
- Dropped requests detected
- Cache storage approaching capacity
- Significant cache hit ratio degradation

Version control: Most Varnish Prometheus exporters are community-maintained. Pin the selected image to an exact version and review it before production use.

9. NGINX Monitoring

NGINX runs inside the WordPress container and communicates with PHP-FPM. Enable the standard NGINX stub_status module and run NGINX Prometheus Exporter as a separate Docker Compose service.

Recommended deployment

- NGINX remains inside the WordPress container
- NGINX Prometheus Exporter runs as a separate Compose service
- The exporter connects to stub_status through the internal Compose network
- stub_status is not exposed publicly
- Prometheus accesses only the exporter metrics port

Metrics

- Active, accepted, and handled connections
- Total request count
- Reading, writing, and waiting connections

Limitation: Standard NGINX stub_status does not provide HTTP status-code or request-latency metrics. Collect those from Traefik, NGINX access logs, or Blackbox Exporter.

10. WordPress and PHP-FPM Monitoring

WordPress does not provide a standard Prometheus metrics endpoint. Monitor availability using the existing homepage, platform metrics, and logs.

- External HTTP probes against the homepage
- Traefik request and error metrics
- NGINX metrics
- PHP-FPM and WordPress logs
- Container resource metrics

PHP-FPM can optionally expose its standard status page through the internal Compose network. A reviewed exporter can provide active and idle workers, listen queue depth, max-children events, slow requests, and worker failures.

Recommended alerts

- PHP-FPM exporter unavailable
- PHP-FPM listen queue increasing or maximum children reached
- PHP slow-request rate increasing
- PHP fatal errors or WordPress database connection errors in logs

Security: The PHP-FPM status page must not be exposed publicly.

11. Centralized Logging

Store logs in Grafana Loki and display them in Grafana. Promtail should not be selected for a new deployment because it reached end of life on March 2, 2026. Use Grafana Alloy instead.

Grafana Alloy can use Docker discovery and loki.source.docker to collect logs from running containers and forward them to Loki.

Collect

- Docker container and Docker daemon logs
- Traefik access and error logs
- Varnish logs
- NGINX access and error logs
- PHP-FPM and WordPress logs
- MySQL error and slow-query logs
- Memcached logs

Recommended labels

- host, compose_project, compose_service
- container, environment

Request IDs, user IDs, URLs, and IP addresses should remain in the log message rather than being used as Loki labels.

Recommended log-based alerts

- Rapid increase in HTTP 5xx responses

- PHP fatal errors or WordPress database connection errors
- MySQL crashes, corruption messages, or sustained slow-query increases
- NGINX upstream connection failures
- Varnish backend fetch failures
- Repeated container startup failures or Docker daemon errors

Docker socket security: Docker socket access provides extensive control over the host. If Alloy uses Docker socket discovery, mount the socket read-only and harden the Alloy container.

12. Prometheus Target Configuration

When Prometheus runs in the same Compose environment, targets can use Compose service names:

```
cadvisor:8080
node-exporter:9100
mysqld-exporter:9104
memcached-exporter:9150
nginx-exporter:9113
varnish-exporter:9131
traefik:8082
```

When Prometheus runs on another server, exporter ports should be:

- Bound only to the Docker host's private IP address
- Protected by firewall rules
- Accessible only from the monitoring server or VPN
- Never exposed directly to the internet

Alternatively, run Grafana Alloy or Prometheus Agent locally and send metrics to the client's central Prometheus-compatible backend using `remote_write`.

13. Alerting and Dashboards

Prometheus should evaluate metric-based alert rules. Alertmanager should handle notification routing, grouping, inhibition, silences, and delivery to email, Slack, Teams, or PagerDuty.

Recommended Grafana dashboards

1. Public website availability and TLS
2. Docker host resources
3. Docker container resources
4. Traefik HTTP traffic
5. MySQL
6. Memcached
7. Varnish
8. NGINX and PHP-FPM
9. Centralized application logs
10. Monitoring target availability

Incident grouping: A MySQL failure may generate WordPress errors, Traefik HTTP 5xx responses, and exporter alerts. Group related signals into one incident where possible.

Recommended Minimum Implementation

1. Prometheus, Alertmanager, and Grafana on a separate monitoring server
2. Blackbox Exporter checking the public WordPress homepage
3. Node Exporter on the Docker host
4. cAdvisor on the Docker host
5. Native Traefik Prometheus metrics
6. mysqld_exporter
7. Memcached exporter
8. Varnish exporter based on varnishstat
9. NGINX Prometheus Exporter as a separate Compose service
10. Loki
11. Grafana Alloy for Docker log collection
12. Alerts for missing containers, resource exhaustion, service failures, HTTP errors, and certificate expiration

Private access only: Exporter ports and metrics endpoints must remain private. Only the public WordPress website should be reachable from the internet.

References

1. Prometheus Blackbox Exporter guide
<https://prometheus.io/docs/guides/multi-target-exporter/>
2. Node Exporter
https://github.com/prometheus/node_exporter
3. cAdvisor Prometheus metrics
<https://github.com/google/cadvisor/blob/master/docs/storage/prometheus.md>
4. Docker Engine Prometheus metrics
<https://docs.docker.com/engine/daemon/prometheus/>
5. Traefik Prometheus metrics
<https://doc.traefik.io/traefik/reference/install-configuration/observability/metrics/>
6. MySQL Prometheus Exporter
https://github.com/prometheus/mysqld_exporter
7. Prometheus exporter catalog
<https://prometheus.io/docs/instrumenting/exporters/>
8. Varnish statistics
<https://varnish-cache.org/docs/7.6/users-guide/operation-statistics.html>
9. NGINX Prometheus Exporter
<https://github.com/nginx/nginx-prometheus-exporter>
10. Grafana Alloy Docker log collection
<https://grafana.com/docs/loki/latest/get-started/quick-start/tutorial/>
11. Promtail end-of-life notice
<https://grafana.com/docs/loki/latest/send-data/promtail/>